

```

function [v1,v2] = universalvar(r1vec,r2vec,dt,way)

mu = 1; % canonical
r1 = norm(r1vec);
r2 = norm(r2vec);
err = 1e-6; % convergence criterion
diff = 1;
i = 0;
maxit = 100;

% solve for delta nu long vs short way
dnu = dot(r1vec,r2vec)/(r1*r2);
if strcmp(way,'long')
    dnu = 2*pi - dnu;
end

% solve for A
A = sqrt(r1*r2)*sin(dnu)/(sqrt(1-cos(dnu)));

while diff > err && i < maxit
    i = i+1;
    % pick a trial value for z
    z = 0.2;

    % evaluate s and c at that point
    % using function from last problem
    [C, S] = calcCandS(z);

    % solve for y
    y = r1 + r2 - ((A*(1-z*S))/(sqrt(C)));

    % solve for x
    x = sqrt(y/C);

    % solve for deltat and compare
    dt_calc = (x^3*S + A*sqrt(y)) / sqrt(mu);

    % UPDATING Z USING NEWTON METHOD
    if z ~= 0
        dCdz = (1/(2*z))*(1-(z*S)-(2*C));
        dSdz = (1/(2*z))*(C-(3*S));
    else
        dCdz = 0;
        dSdz = 0;
    end

    % dy/dz
    dydz = -A/C * (sqrt(C)*(((1-S-(z*dSdz)))-(((1-z*S)/(2*sqrt(C)))*dCdz)));

    % dx/dz
    dxdz = (1/(2*x*C))*(dydz-(x^2*dCdz));

    % ddt/dz
    ddt_dz = (1/mu)*((3*x^2*dxdz*S + (x^3*dSdz) + ((A/(2*sqrt(y)))*dydz)));

    % Newton-Raphson step
    dz = (dt_calc - dt) / ddt_dz;
    z = z - dz;

```

```

% Update convergence
diff = abs(dt_calc - dt);

fprintf('Iter %d: z = %.10f,delta = %.2e\n',i,z,diff);

end

% use final z to calculate s and z again
[C, S] = calcCandS(z);
y = r1 + r2 + A * (z * S - 1) / sqrt(C);

% now calculating f and g
f = 1 - y/r1;
g = A*sqrt(y/mu);
gdot = 1-y/r2;

% Velocity vectors
v1 =(r2vec-f*r1vec)/g;
v2 =(gdot*r2vec-r1vec)/g;

end

```

Not enough input arguments.

Error in universalvar (line 4)
r1 = norm(r1vec);